

Analiza zużycia energii urządzenia Zigbee End Device na platformie nRF54L15

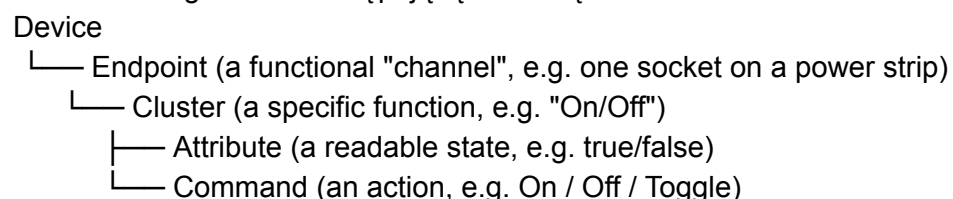
Cel: projekt polegał na skonfigurowaniu sieci Zigbee z wykorzystaniem nRF52840 dongle jako koordynatora sieci i nRF54L15 jako sleepy end device. Na koniec zostały przeprowadzone badania wpływu konfiguracji parametrów transmisji na zużycie energii. Szukaliśmy optymalnych parametrów, które zapewniają najniższy pobór prądu. Pomiary zostały zrealizowane za pomocą Power Profiler Kit II (PPK2).

Czym jest sieć Zigbee?

Zigbee jest protokołem bezprzewodowym, który jest wykorzystywany w systemach IoT inteligentnych budynków. Zaletą urządzeń w sieci Zigbee jest to, że zużywają mało energii i mogą długo działać na baterii. Sieć Zigbee składa się z trzech ról: koordynator, router i urządzenie końcowe.

- **Koordynator:** jeden na całą sieć, jego zadaniem jest stworzenie sieci i zarządzanie nią. Dodatkowo działa jako Trust Center, jest odpowiedzialny za nadawanie kluczy sieciowych, gdy nowe urządzenie dołącza do sieci.
- **Router:** zasilany energią elektryczną, ma zawsze uruchomione radio, więc nasłuchuje nieprzerwanie i może odebrać wiadomość w dowolnym momencie; w sieci jest odpowiedzialny za przesyłanie ruchu dalej, wzmacniając sieć.
- **End device:** zazwyczaj zasilane bateryjnie (czujniki, urządzenia uśpione). Komunikuje się tylko za pośrednictwem nadrzędnego koordynatora lub routera i może przechodzić w tryb uśpienia między transmisjami, aby oszczędzać energię.

Każde urządzenie w sieci Zigbee ma następującą strukturę:



Urządzenie może mieć kilka endpointów (zwykle ma jeden). Endpoint jest kanałem, który zawiera klastry „funkcje” danego urządzenia (np. on/off, temperature-measurement). Atrybut to wartość opisująca dany klaster, a komenda to funkcja pozwalająca wywołać akcję na danym klastrze.

Koordynator w naszej sieci Zigbee miał jeden endpoint i trzy klastry (dwa wejściowe i jeden wyjściowy). Klaster wejściowy to informacja, którą ktoś może poprosić od koordynatora, a

koordynator odpowiada odpowiednio wypełnionymi atrybutami, koordynator ma klastry wejściowe Basic (informacje o urządzeniu, takie jak model i wersja oprogramowania) oraz Identify (umożliwia innym urządzeniom komunikację z urządzeniem). Klaster wyjściowy to coś, co koordynator może wysłać - w naszym przypadku koordynator wysyła żądanie włączenia/wyłączenia światła (klaster wyjściowy on/off).

Zigbee ma dwie warstwy protokołu:

- ZCL (Zigbee Cluster Library) warstwa aplikacyjna: komendy typu Toggle, odczyt/zapis atrybutów.
- ZDO (Zigbee Device Object) warstwa zarządzania siecią i odkrywania urządzeń.

Koordynator

Koordynator został zaimplementowany na płytce nRF52840 dongle. Najważniejszym zadaniem po stronie koordynatora jest stworzenie sieci oraz komunikacja z end device i routerem. *zboss_signal_handler* jest metodą wywoływaną automatycznie przez ZBOSS(implementacja stosu protokołu Zigbee, który obsługuje całą logikę sieciową) za każdym razem, gdy nastąpi nowe zdarzenie, np. utworzenie sieci lub dołączenie nowego urządzenia, po czym kod decyduje, co dalej z nim zrobić. Gdy koordynator uruchamia się z pustą pamięcią NVRAM (nigdy wcześniej nie tworzył sieci), generowany jest sygnał, który przekazuje stosowi: „utwórz zupełnie nową sieć”.

Stworzenie sieci wygląda następująco:

1. **Skanowanie kanałów** - urządzenie skanuje wszystkie 16 kanałów jeden po drugim, mierząc szum tła na każdym z nich, a następnie osiada na najcichszym. Cichy kanał ma znaczenie, ponieważ zajęty kanał oznacza rywalizację o czas antenowy z innymi sieciami lub zakłócenia, co prowadzi do kolizji, uszkodzonych pakietów i retransmisji - cichy kanał utrzymuje fale radiowe wolne dla ruchu w tej sieci. Ten wybór jest zapisywany w pamięci NVRAM; aby wymusić nowe skanowanie, wymagane jest zresetowanie pamięci urządzenia.
2. **Generowanie identyfikatora PAN** - koordynator generuje 16-bitowy identyfikator sieci, sprawdzając, czy nie jest on już używany przez inną sieć. W tym momencie sieć technicznie istnieje, ale nadal jest zamknięta.
3. **Utrwalanie w pamięci NVRAM** - identyfikator PAN i kanał są zapisywane w pamięci flash, więc ponowne uruchomienie ponownie ładuje istniejącą sieć, zamiast tworzyć nową.

Te kroki nazywane są formacją i polegają na wyborze kanału oraz PAN ID. Następnym krokiem po formacji jest steering, na tym etapie koordynator otwiera sieć dla nowych urządzeń. Sieć otwiera się na 180 s; takie okno nazywa się permit join. **Permit join** to licznik czasu uruchamiany przez koordynatora po wywołaniu funkcji steering: przez 180 sekund sieć

akceptuje nowe urządzenia próbujące się do niej dołączyć. Po zamknięciu tego okna sieć działa normalnie, ale żadne nowe urządzenie nie może się do niej dołączyć, dopóki okno nie zostanie otwarte ponownie.

Metoda `zboss_signal_handler` obsługuje następujące zdarzenia:

- **FIRST START** urządzenie właśnie się uruchomiło -utwórz nową sieć Zigbee, ZBOSS wybiera kanał i identyfikator PAN, tworzy sieć.
- **FORMATION** sieć została pomyślnie utworzona - otwórz sieć, aby umożliwić dołączanie.
- **REBOOT** urządzenie zostało zrestartowane, ale sieć już istnieje - otwórz sieć, aby umożliwić dołączanie.
- **STEERING** sieć jest otwarta na dołączanie -włącz diodę LED, wskazując, że sieć jest gotowa.

Sieć zostanie utworzona i uruchomiona automatycznie od razu po wywołaniu funkcji `zigbee_enable()`.

Teraz koordynator tworzy sieć i pozwala urządzeniom dołączać, ale nie ma pojęcia, kto dołączył więc naszym celem jest rozpoznanie tego urządzenia. Do `zboss_signal_handler` zostały dodane dwa zdarzenia obsługujące dołączanie nowego urządzenia:

- **DEVICE ANNCE** - gdy nowe urządzenie dołącza do sieci Zigbee, powinno nadać komunikat („jestem tutaj, to mój adres”).
- **DEVICE AUTHORIZED** - sygnał odbierany przez aplikację Zigbee, gdy nowe urządzenie zakończy uzgadnianie zabezpieczeń, Trust Center wymienia z nim klucze, a urządzenie staje się zabezpieczone w sieci.

Dlaczego zabezpieczać? Przed zabezpieczeniem urządzenie końcowe dołącza do sieci, ale nie może poprawnie komunikować się z nikim w zaufany sposób. Trust Center nadaje urządzeniu klucz sieciowy (rodzaj współdzielonego hasła, którego wszystkie urządzenia w sieci używają do szyfrowania/odszyfrowywania wiadomości). Bez klucza wiadomość byłaby bezsensowna.

Po podłączeniu urządzenia końcowego do sieci możemy sterować nim od strony koordynatora. Nie możemy jednak od razu wysłać sygnału przełączającego, ponieważ nie wiemy, który endpoint żarówki ma klaster on/off. Dlatego wykonujemy ZDO discovery, zadając trzy zapytania. ZDO discovery polega na tym, że koordynator pyta nowo dołączone urządzenie: „kim jesteś i co potrafisz?”.

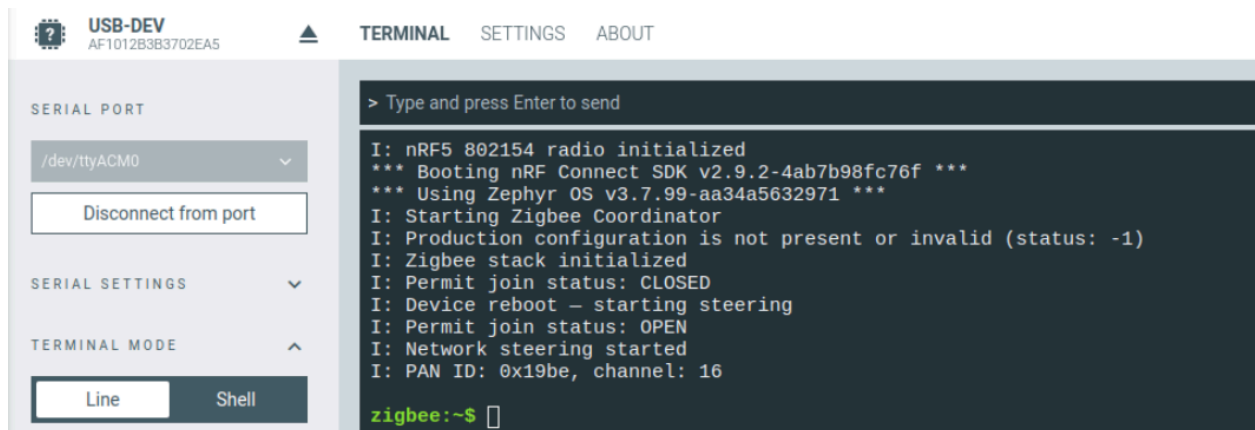
The discovery chain - trzy pytania ZDO zadawane po kolei:

1. **Active_EP_req** - wyświetl wszystkie numery endpointów, które posiadasz (np. endpoint 1 = żarówka, endpoint 2 = przycisk).

2. **Simple_Desc_req** - wysyłane dla każdego endpointu z tej listy, aby sprawdzić, jakie klastry (funkcje) obsługuje każdy endpoint (np. klaster on/off, klaster temperatury).
3. **Bind_req** - łączy endpoint urządzenia z endpointem i klastrem docelowym, aby polecenia wiedziały, gdzie mają trafić. Gdy znajdziemy endpoint z opcją on/off, tworzymy binding: mój endpoint 10 = twój endpoint 20, który ma klaster on/off.

Po tym jak binding zakończył się sukcesem, możemy sterować diodą LED za pomocą przycisku. Binding odblokowuje możliwość wysyłania poleceń.

Następnie koordynator został rozszerzony o shell. Shell to interpreter wiersza poleceń służący do komunikacji z urządzeniem poprzez wpisywanie poleceń. Czeki na dane wejściowe, sprawdza, czy rozpoznaje wpisane polecenie i uruchamia odpowiedni kod. Sposób, w jaki wpisywanie w terminalu dociera do układu: port COM jest siecią dwukierunkową (układ → komputer i komputer → układ). Ustawienie CONFIG_SHELL_BACKEND_SERIAL=y powoduje, że shell nasłuchuje znaków przychodzących na tym samym porcie szeregowym UART/USB. W programie zostały zdefiniowane komendy: toggle (do włączania/wyłączania światła), open (aby otworzyć sieć, gdy jest zamknięta po permit join), devices (wyświetla listę podłączonych urządzeń), name (pozwala nadawać nazwy urządzeniom).



Rysunek 1: Sieć stworzona przez koordynatora

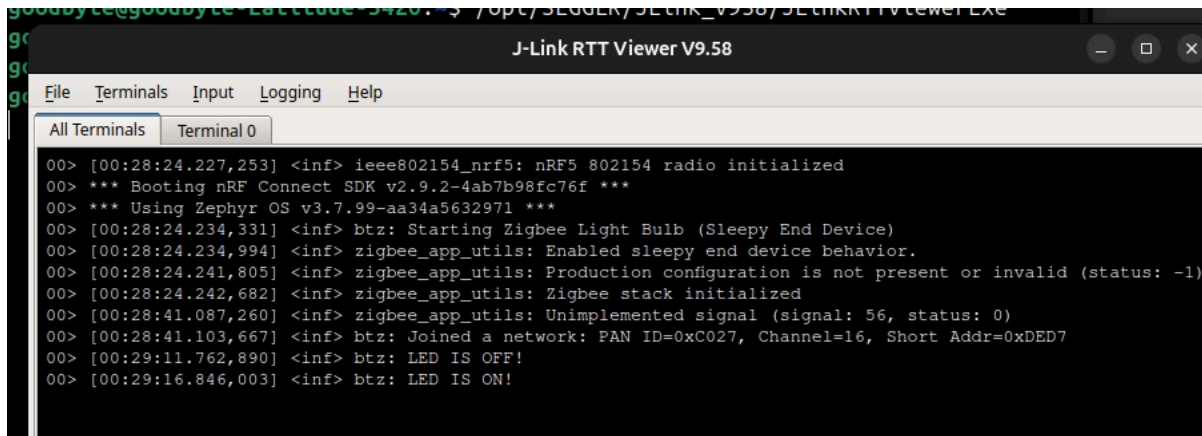
Router

Router został zaimplementowany na płytce nRF54L15-DK. Router nie jest wykorzystywany w wersji końcowej i został zbudowany w celach edukacyjnych. Struktura routera jest prawie taka sama jak struktura end device, więc stworzenie programu dla routera było pomocne przy tworzeniu programu dla end device.

Po stronie routera znajduje się druga funkcja `zboss_signal_handler`, która obsługuje następujące zdarzenia:

- **FIRST START** - urządzenie nie zna żadnej sieci, więc zaczyna szukać otwartych sieci i próbuje się do nich dołączyć.
- **REBOOT** - urządzenie już zna tę sieć, więc podłącza się do znanej sieci.
- **STEERING** - urządzenie jest już podłączone do sieci i ma swój własny adres w sieci.

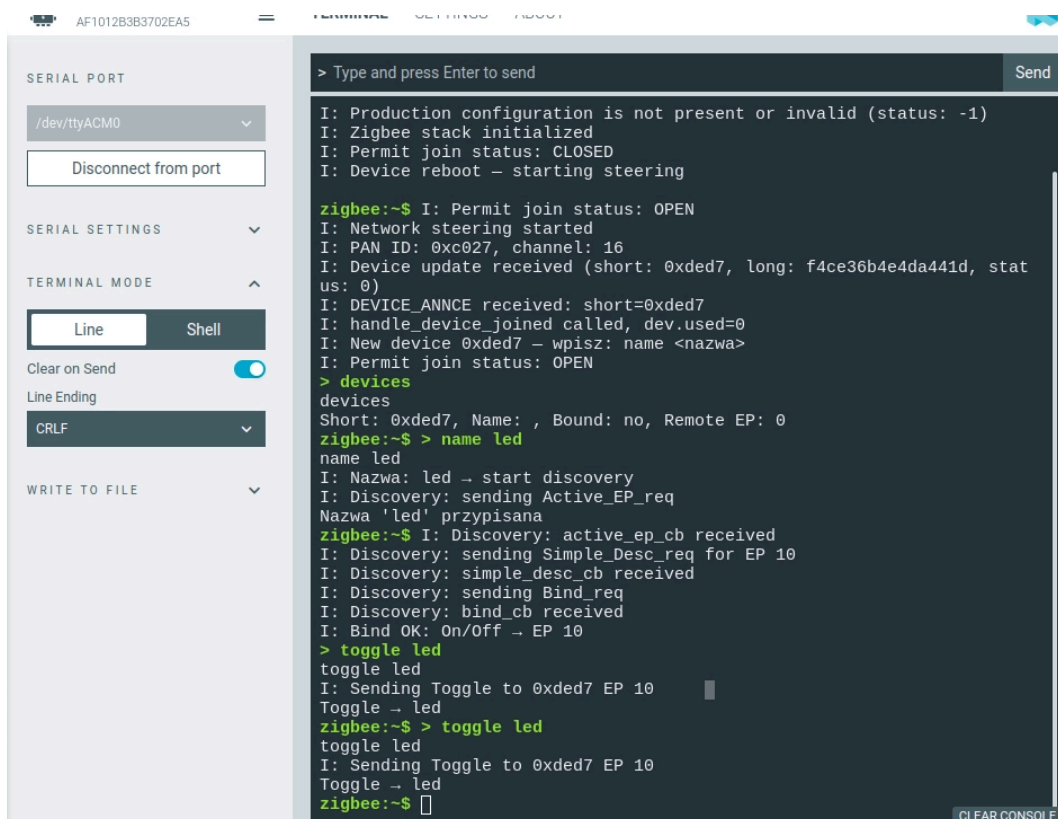
Router otrzymuje sygnał, że koordynator chce zarządzać światłem; ZBOSS obsługuje ten sygnał i zmienia stan światła lampki.



```

00> [00:28:24.227,253] <inf> ieee802154_nrf5: nRF5 802154 radio initialized
00> *** Booting nRF Connect SDK v2.9.2-4ab7b98fc76f ***
00> *** Using Zephyr OS v3.7.99-aa34a5632971 ***
00> [00:28:24.234,331] <inf> btz: Starting Zigbee Light Bulb (Sleepy End Device)
00> [00:28:24.234,994] <inf> zigbee_app_utils: Enabled sleepy end device behavior.
00> [00:28:24.241,805] <inf> zigbee_app_utils: Production configuration is not present or invalid (status: -1)
00> [00:28:24.242,682] <inf> zigbee_app_utils: Zigbee stack initialized
00> [00:28:41.087,260] <inf> zigbee_app_utils: Unimplemented signal (signal: 56, status: 0)
00> [00:28:41.103,667] <inf> btz: Joined a network: PAN ID=0xC027, Channel=16, Short Addr=0xDED7
00> [00:29:11.762,890] <inf> btz: LED IS OFF!
00> [00:29:16.846,003] <inf> btz: LED IS ON!
  
```

Rysunek 2: Jak router wyświetla dołączenie do sieci koordynatora i jak reaguje na jego żądania



```

> Type and press Enter to send
Send

I: Production configuration is not present or invalid (status: -1)
I: Zigbee stack initialized
I: Permit join status: CLOSED
I: Device reboot - starting steering

zigbee:~$ I: Permit join status: OPEN
I: Network steering started
I: PAN ID: 0xc027, channel: 16
I: Device update received (short: 0xded7, long: f4ce36b4e4da441d, status: 0)
I: DEVICE_ANNCE received: short=0xded7
I: handle_device_joined called, dev.used=0
I: New device 0xded7 - wpisz: name <nazwa>
I: Permit join status: OPEN
> devices
devices
Short: 0xded7, Name: , Bound: no, Remote EP: 0
zigbee:~$ > name led
name led
I: Nazwa: led - start discovery
I: Discovery: sending Active_EP_req
Nazwa 'led' przypisana
zigbee:~$ I: Discovery: active_ep_cb received
I: Discovery: sending Simple_Desc_req for EP 10
I: Discovery: simple_desc_cb received
I: Discovery: sending Bind_req
I: Discovery: bind_cb received
I: Bind OK: On/Off - EP 10
> toggle led
toggle led
I: Sending Toggle to 0xded7 EP 10
Toggle - led
zigbee:~$ > toggle led
toggle led
I: Sending Toggle to 0xded7 EP 10
Toggle - led
zigbee:~$
  
```

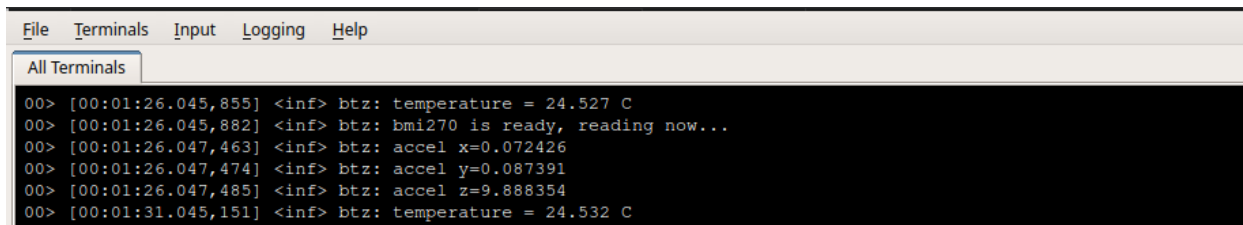
Rysunek 3: Koordynator stworzył sieć i teraz możemy mieć interakcję z routerem

End device

Implementacja jest zasadniczo taka sama jak w przypadku routera, różnica między nimi polega nie na strukturze kodu, ale na zachowaniu:

- Router jest zawsze zasilany, a jego radio jest zawsze włączone. Urządzenie końcowe jest uśpione prawie cały czas i budzi się tylko na czas odpytywania.
- Router nasłuchuje nieprzerwanie i może odebrać wiadomość w dowolnym momencie. Radio urządzenia końcowego jest wyłączone przez większość czasu, może ono odebrać wiadomość dopiero po zapytaniu urządzenia nadrzędnego „czy jest coś dla mnie?”.

End device ma trzy czujniki: czujnik położenia, temperatury i jasności. Zaimplementowane zostały czujnik położenia i temperatury.

A screenshot of a terminal window with a menu bar (File, Terminals, Input, Logging, Help) and a tab labeled 'All Terminals'. The terminal displays a series of log messages from a device. The messages show timestamps in brackets followed by an information log entry. The data includes temperature readings (24.527 C, 24.532 C) and acceleration data (x=0.072426, y=0.087391, z=9.888354).

```
00> [00:01:26.045,855] <inf> btz: temperature = 24.527 C
00> [00:01:26.045,882] <inf> btz: bmi270 is ready, reading now...
00> [00:01:26.047,463] <inf> btz: accel x=0.072426
00> [00:01:26.047,474] <inf> btz: accel y=0.087391
00> [00:01:26.047,485] <inf> btz: accel z=9.888354
00> [00:01:31.045,151] <inf> btz: temperature = 24.532 C
```

Rysunek 4: Wartości które udało się odczytać z czujników

Testowanie

Wszystkie pomiary zostały wykonane za pomocą aplikacji **board-power-test**, która steruje całym cyklem bez udziału operatora: kompiluje firmware z zadanymi parametrami, flashuje płytkę przez J-Link, włącza i wyłącza zasilanie z PPK2, czeka na potwierdzenie dołączenia urządzenia do sieci, rejestruje prąd i zapisuje wynik do współdzielonego dziennika (reports/pomiary.csv) wraz z surowym przebiegiem.

Aplikacja istniała już w stanie roboczym na początku tych prac. Została zmieniona pod potrzeby Zigbee za pomocą Claude Code.

Przed rozpoczęciem pomiarów wartość pobieranego prądu wynosiła około 200 μ A, co było zbyt dużo. Po zbadaniu kodu okazało się, że przyczyną tak wysokiego poboru prądu były: stale włączona dioda, aktywne czujniki, logowanie do konsoli oraz nieodpowiednie ustawienia konfiguracyjne. Po wyłączeniu wszystkich tych przyczyn pobór prądu spadł do ~ 10 μ A.

W sieci Zigbee dla sleepy end device ręcznie ustawiamy parametry następujących funkcji:

- **zigbee_configure_sleepy_behavior(true)** ustawia rx_on_when_idle na false. Bez tego parametru odbiornik pozostaje włączony na stałe, a urządzenie pobiera miliampery

niezależnie od pozostałych ustawień. To właśnie ten przełącznik umożliwia niskie zużycie energii.

- **Poll interval** określa, jak często urządzenie pyta swojego rodzica „czy masz coś dla mnie?”. Ponieważ radio jest wyłączone, jest to jedyny sposób, w jaki uśpione urządzenie może odbierać dane w łączu w dół. Każde takie zapytanie wymaga wybudzenia radia, więc ten parametr jest bezpośrednio przeliczany na pobierany prąd i ma najszerszy użyteczny zakres (od 1 s do ~20 dni).

Ważne: wartość musi zostać ustawiona po dołączeniu urządzenia do sieci. W trakcie procedury dołączania stos resetuje ją do wartości domyślnej (5 s). Ustawienie tej opcji przed wywołaniem *zigbee_enable()* powoduje, że urządzenie odpytuje co 5 sekund, czyli około 720 wybudzeń na godzinę zamiast jednego co przekłada się na pobór prądu na poziomie ~4,6 μ A. W tym projekcie ustawienie to jest więc stosowane w procedurze obsługi sygnału SIGNAL STEERING.

- **Keepalive timeout** określa częstotliwość, z jaką urządzenie potwierdza swojemu rodzicowi, że wciąż jest aktywne. W ZBOSS realizowane jest to za pomocą tego samego odpytywania danych MAC, dokumentacja API opisuje ten parametr jako „jak często urządzenie powinno odpytywać swojego rodzica”. Nie generuje on osobnego kosztu energetycznego; jedynie ustala dolną granicę częstotliwości odpytywania.
- **End device timeout** to czas, który urządzenie negocjuje ze swoim rodzicem. Rodzic uruchamia odliczanie dla każdego dziecka i usuwa je, jeśli w tym czasie nic od niego nie odbierze. Sam parametr nie pobiera energii, ale ustala sztywne ograniczenie częstotliwości, z jaką urządzenie musi się rejestrować. Utrata połączenia jest kosztowna: urządzenie musi ponownie się połączyć, co wiąże się z krótkotrwałym, ale wysokim poborem prądu.

Z tych czterech parametrów największy wpływ na pobór prądu mają wartość poll interval oraz częstotliwość wysyłania wiadomości do koordynatora. Z tego powodu pomiary zostały wykonane w dwóch konfiguracjach:

- stała wartość poll interval i zmienna częstotliwość wysyłki,
- zmienna wartość poll interval i zmienna częstotliwość wysyłki.

Badania zostały przeprowadzone dla wartości $T = 10, 20, 30, 60, 120, 300, 600$ [s]. Każda seria została zmierzona dwukrotnie, a zgłoszona wartość jest średnią z obu przebiegów.

Naszym celem było ograniczenie liczby polli, zarówno w celu zmniejszenia zużycia prądu, jak i zachowania czystości zarejestrowanego przebiegu, aby możliwe było zidentyfikowanie poszczególnych zdarzeń i zintegrowanie ich ładunku.

Odkrycie: poll i keepalive to w praktyce ten sam komunikat. Odpowiadają na różne pytania - poll dotyczy tego, jak szybko chcemy odbierać dane, a keepalive dotyczy tego, jak często musimy udowodnić, że jesteśmy aktywni, ale oba warunki spełnia jedno i to samo odpytywanie danych MAC. Konsekwencją tego jest to, że urządzenie wybudza się zgodnie z tym z dwóch interwałów, który jest krótszy. Podniesienie tylko jednego z nich nic nie zmienia, ponieważ drugi staje się nowym progiem ograniczającym. Z tego powodu oba ustawiliśmy na tę samą wartość.

Oba parametry ustawiliśmy na 3600 s, ponieważ wartość ta spełnia dwa niezależne ograniczenia:

- **Dolna granica (bezpieczeństwo):** limit czasu starzenia (aging timeout) wynosi 256 min; odporność na chwilowe zakłócenia wymaga ≥ 3 potwierdzeń aktywności w tym oknie $\rightarrow 256 / 3 \approx 85$ min.
- **Górna granica:** najdłuższy testowany interwał wysyłania wynosi 3600 s (60 min); odpytywanie rzadziej niż wysyłanie danych niczego nie oszczędza, ponieważ urządzenie i tak jest wtedy aktywne.

Wartość 3600 s spełnia oba warunki: daje 4,3 potwierdzenia na okno starzenia (znacznie powyżej wymaganego minimum) i nie powoduje żadnego dodatkowego wybudzenia, którego nie wymuszałby już harmonogram wysyłania danych. Warto zaznaczyć, że nie jest to maksymalna możliwa wartość — interwał odpytywania można legalnie ustawić na ok. 20 dni, co jednak spowodowałoby usunięcie urządzenia z sieci w ciągu ok. 4 godzin. Aby zapewnić odpowiedni margines bezpieczeństwa, limit czasu starzenia (end device aging) został zwiększony do 256 min (15360 s).

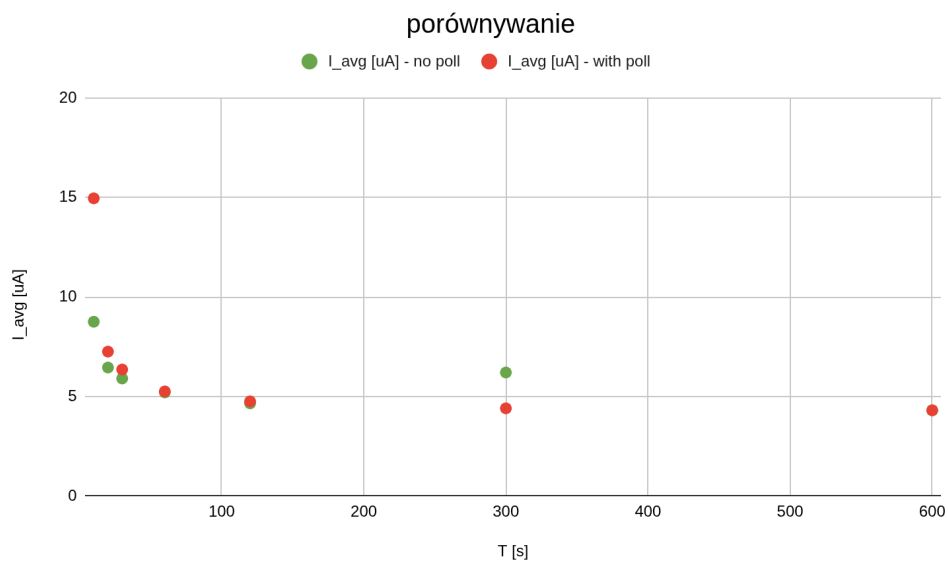
T [s]	CONFIG_ZB_SEND_INTERVAL_S [s]	Czas pomiaru [min]	I1 [uA]	I2 [uA]	I_avg [uA]
10	10	5	8.7	8.8	8.75
20	20	10	6.4	6.5	6.45
30	30	15	6.1	5.7	5.9
60	60	25	5.4	5	5.2
120	120	40	4.7	4.6	4.65
300	300	100	6.2		6.2
600	600	200	4.3		4.3

Tabela 1: Wartość poll stała i wynosi 3600s, zmieniamy tylko częstotliwość wysyłki

T [s]	CONFIG_ZB_SEND_INTERVAL_S [s]	CONFIG_ZB_POLL_INTERVAL_S [s]	Czas pomiaru [min]	I1 [uA]	I2 [uA]	I_avg [uA]
10	10	10	5	19.1	10.8	14.95
20	20	20	10	7.1	7.4	7.25
30	30	30	15	6.3	6.4	6.35
60	60	60	25	5.2	5.3	5.25
120	120	120	40	4.8	4.7	4.75
300	300	300	100	4.4		4.4
600	600	600	200	4.3		4.3

Tabela 2: Wartość poll i częstotliwość wysyłki są takie same

Im dłuższy jest interwał pollingu, tym bardziej krzywa się spłaszcza i pobór prądu jest najmniejszy. Gdy poll wysyłany jest często, zużywa dużo prądu, bo wymusza częste wybudzanie radia, dlatego przy T=10s (poll co 10s) widzimy największą różnicę w pomiarach. To pokazuje, że sam polling zużywa dodatkową energię, gdy odbywa się tak często, bo radio musi się ciągle włączać, żeby zapytać: „czy są dla mnie jakieś wiadomości?”.



Rysunek 5: Porównanie zużycia prądu ze stałym interwałem poll (3600 s) i zmienną częstotliwością wysyłki + zmienną wartością poll

Już od T > 20s różnica spada i krzywa zbiega do wartości ~4,4 μ A, to dolna granica którą reprezentuje prąd spoczynkowy urządzenia plus stały narzut samego stosu Zigbee, którego nie da się wyłączyć. Oznacza to, że gdy interwał jest wystarczająco długi, dodatkowe wybudzenia nie kosztują już wiele, cykle są rozłożone w czasie na tyle rzadko, że nie zmieniają sumarycznego zużycia w istotny sposób. Przejście z 300 s na 600 s daje zysk tylko 0,1 μ A, więc

nie ma sensu dalej wydłużać interwału pollingu. Najlepszy kompromis między zużyciem energii a responsywnością znajduje się gdzieś pomiędzy 120 s a 300 s - dalsze skracanie interwału nie jest uzasadnione, bo koszt energetyczny rośnie znacznie szybciej niż realna korzyść z częstszej komunikacji.

Rozszerzenie aplikacji board-power-test o „kalkulator poboru prądu” pozwoliło, zamiast fizycznego pomiaru każdego interwału, przewidywać średni prąd dla dowolnego interwału. Model jest liniowy w $1/T$ i identyczny dla wszystkich trzech protokołów, wykorzystując wzór:

$$I_{avg} = I_{baseline} + \sum (Q_i / T_i)$$

Formuła ta została rozszerzona o dodatkowy człon Q_ZBOSS/T_ZBOSS , ponieważ w transmisji istnieje niezależne od pollingu zdarzenie, cykliczny sygnał zegara wewnętrznego ZBOSS, który pobiera stały prąd $0,92 \mu A$ i bez którego model systematycznie zaniżał realny pobór energii, szczególnie przy długich interwałach T .

Deep sleep to stan, w którym układ gasi rdzeń i traci zawartość RAM; wybudzenie budzikiem GRTC jest pełnym resetem, więc po każdym cyklu stos ZBOSS wstaje od zera i musi ponownie dołączyć do sieci. Po przetestowaniu przy ładunku $3500 \mu C$ na jeden cykl (boot + ponowne dołączenie + wysłanie) deep sleep opłaca się dopiero powyżej ok. 900 s (15 min), natomiast przy krótszych odstępach reboot i ponowne łączenie z siecią kosztują więcej energii niż udaje się zaoszczędzić.

Wnioski

Z wykresu wynika, że dla tego samego zadania BLE Mesh zużywa najmniej energii, Thread jest pośrodku, a Zigbee zużywa najwięcej. Ta różnica jest widoczna praktycznie na każdym testowanym interwale, od 10 s do 600 s.

Dlaczego BLE Mesh wygrywa: BLE Mesh od początku był projektowany z myślą o urządzeniach bateryjnych i ma bardzo lekki protokół komunikacyjny - mniej narzutu na każdą wiadomość, prostszy stos. Dlatego przy każdym interwale BLE Mesh pobiera najmniej prądu.

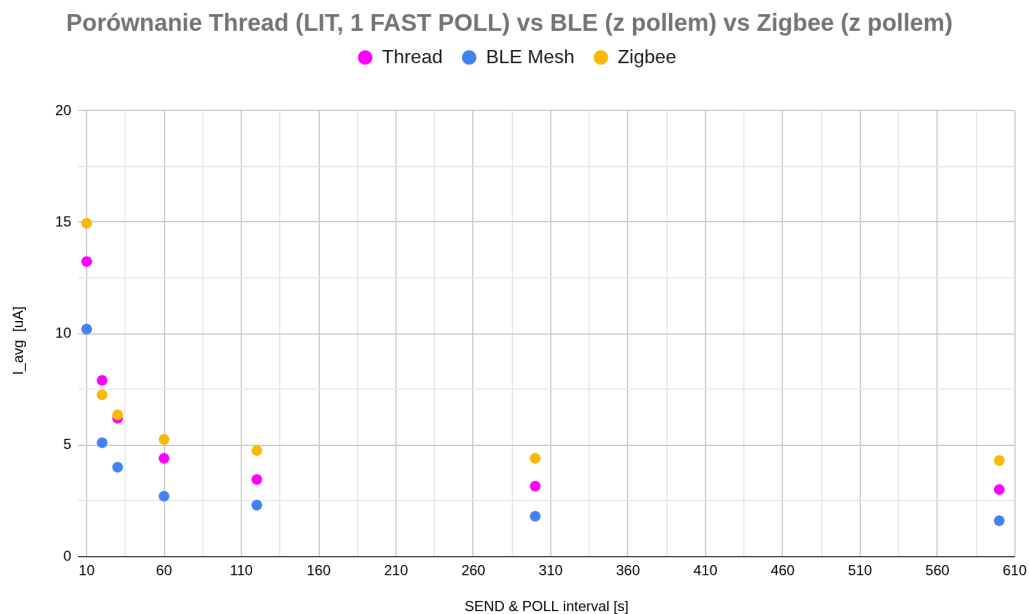
Dlaczego Zigbee wypada najslabiej: wyniki Zigbee można uzasadnić utratą prądu na zegar ZBOSS, który pobiera stały prąd $0,92 \mu A$, a także tym, że Zigbee jest „trudniejszym” protokołem, który ma więcej danych w każdym pakiecie powoduje wyższe zużycie energii przy każdej transmisji, niezależnie od interwału pollingu.

Czy Zigbee można poprawić? Podczas pomiarów już znacznie zmniejszyliśmy pobór prądu i to właśnie spowodowało Zigbee z $\sim 200 \mu A$ do poziomu widocznego na wykresie. Dalej jednak nie da się zejść, heartbeat ZBOSS jest zaszyty w bibliotece i nie da się go wyłączyć, więc stanowi twardy dolny limit tej implementacji.

Zastosowanie: taki profil zużycia energii dobrze nadaje się do urządzeń, które raportują okresowo, a nie reagują natychmiastowo, np. czujniki środowiskowe, wykrywanie obecności, stanu drzwi/okien, wycieków wody czy monitoring zasobów, gdzie opóźnienie rzędu minut jest akceptowalne. Nie nadaje się jednak do aktuatorów (oświetlenie, zawory, przekaźniki) ponieważ niski pobór energii wymaga długiego interwału pollingu (tu: 3600 s), co oznacza, że komenda od koordynatora może czekać nawet godzinę, zanim urządzenie ją odbierze.

Podsumowując: pierwsze pomiary na płytce wynosiły ok. 200 μA , co wynikało z ustawień domyślnych: aktywna dioda, włączona linia zasilania czujników oraz konfiguracja niepozwalająca urządzeniu prawidłowo zasypiać. Po wyłączeniu nieużywanych peryferiów zużycie spadło do poziomu 4,3–8,75 μA . Osiągnięty wynik należy uznać za dobry dla urządzenia typu sleepy end device. Wartość 4,3 μA jest bliska teoretycznemu minimum wynikającemu z samego układu i stosu. Zużycie rozkłada się na: prąd spoczynkowy ($\sim 3,2 \mu\text{A}$), stały narzut stosu ZBOSS (0,92 μA) oraz koszt samego raportowania (0,15 μA przy $T=300\text{s}$).

Przełożenie tego na żywotność baterii daje bardzo praktyczny wniosek: na pojedynczej baterii CR2032 urządzenie może działać od 2 do prawie 5 lat, w zależności od wybranego interwału raportowania.



Rysunek 6: Porównanie zużycia prądu dla Thread, BLE Mesh i Zigbee w funkcji interwału wysyłki i pollingu